

Why Is This Code Bad?

Nathan Jones

Contents

- [Introduction](#)
- [The Problem I Was Trying to Solve](#)
- [My First Solution](#)
- [Attempt #2](#)
- [A Spectrum of Badness](#)
- [The Beauty in Simplicity](#)
- [From Bad to Good](#)
- [Conclusion](#)

Introduction

Four years ago, I found myself working on a problem and was able to arrive at a solution that was much cleaner than my original one. Before I even discuss the problem I was trying to solve, take a look at my two solutions, below. Which would you want to see in a new codebase that you were trying to understand? Which would you want to test or debug? Which would you want to work with if your manager asked you to extend or modify the code?

Option A:

```
def get_dependencies_list(dep_file):
    dep_list = []
    if os.path.exists(dep_file):
        with open(dep_file) as deps:
            line = deps.readline()
            while line:
                if len(line) > 1:
                    while line[-2] == "\\":
                        line = line[:-2] + deps.readline()
                    colon_idx = line.find(': ')
                    if colon_idx != -1:
                        dep_list += line[colon_idx+2:line.find("\n")].split(' ')
                    line = deps.readline()
    for item in dep_list:
        if item == '' or ' ' in item:
            dep_list.remove(item)
    return dep_list
```

Option B:

```
def get_dependencies_list(dep_file):
    dep_list = []
    if file_exists(dep_file):
        with open(dep_file) as f:
            deps = f.read()
            deps = deps.replace('\n', ' ')
            dep_list = deps.split(' ')
            dep_list = list(filter(lambda file: file_exists(file), dep_list))
    return dep_list
```

With one exception, these two functions do the exact same thing. But Option B looks much easier to understand and to work with, and not just because it's the shorter function! The question I had after this realization back then, and the question I'll ask you now, is: "Why? *Why* does Option B seem easier to understand? What, *specifically*, makes it 'better' code?" The answer, I think, may help you write code in the future that not only works, but is also code that you and other developers *actually* want to work with throughout your development lifecycle.

The Problem I Was Trying To Solve

"Write a function that takes in the string name of a dependency file (the same generated by a compiler like GCC during object file compilation; see below) and returns a list/array of strings which represents the filepaths to each of the object file's dependencies."

Build systems like `make` or `ninja` are able to automatically detect when an output file (like `main.o`) needs updating based on how recently the input files (like `main.c`) were edited. However, they can't readily identify when *other* input files, like header files, are updated (which would necessitate rebuilding the output file), since those files aren't explicitly listed as inputs to the build step. However, GCC (and probably every other C compiler there is) is able to generate dependency files during the compilation process that *do* list those implicit dependencies. These files have the format:

```
path/to/output/file: path/to/dependency/1 path/to/dependency/2
```

e.g.

```
build/src/main.o: src/main.c inc/main.h
```

GCC will also use the backslash character to indicate, at the end of a line, when additional dependency files exist below.

e.g.

```
build/src/main.o: src/main.c \  
    inc/main.h
```

Build systems like `make` and `ninja` (and the bespoke one I was writing in Python for fun at the time) can use these dependency files to keep a project up-to-date with *all* input files, even those not explicitly listed in each build step.

For the example above, the function should return `['src/main.c', 'inc/main.h']`.

My First Solution

My first attempt involved reading the file and parsing it to look for `:` and `\`; the colon-space indicates where the first dependency file is (if any) and the backslash indicates when additional dependency files exist on further lines of the file. It looks like this ("Option A", from the introduction):

```
def get_dependencies_list(dep_file):
    dep_list = []
    if os.path.exists(dep_file):
        with open(dep_file) as deps:
            line = deps.readline()
            while line:
                if len(line) > 1:
                    while line[-2] == "\\":
                        line = line[:-2] + deps.readline()
                    colon_idx = line.find(': ')
                    if colon_idx != -1:
                        dep_list += line[colon_idx+2:line.find("\n")].split(' ')
                    line = deps.readline()
            for item in dep_list:
                if item == '' or ' ' in item:
                    dep_list.remove(item)
    return dep_list
```

I basically read the file a line at a time, adding subsequent lines to the current one if it ends in a backslash. I also cut off the last 2 characters in the current line when I do that, to get rid of the backslash and the newline character. Then, I look for the character sequence ": ", indicating the start of the list of dependencies, and I split the line from then until the newline character at the end of the string along spaces. The resulting list may contain extra spaces or empty items, so in the last three lines of the function, I iterate through the whole list and remove any string that doesn't represent a valid file.

Not exactly pretty, eh? Oh, here's the kicker: **there's a bug in the code above**. Can you find it?

Attempt #2

I realized something bad was happening when I was testing this code and the output of the function contained strings that weren't valid files. "What? I thought I checked for those!"

Turns out that I had written the last three lines of code incorrectly. I was iterating over a list in Python *while I was removing elements from that list*. It turns out that the loop variable *always* increments to the next indexed item in the list, *even if a previous item was removed from said list*. Doing so has the effect of "skipping" elements in the list when the previous element is removed. For example, in the list below, if the loop variable is pointing to 'b', and is then removed, the next value of the loop variable is 'd', not 'c'!

1) Loop variable is 'b'

*

['a', 'b', 'c', 'd']

2) 'b' is removed

*

['a', 'c', 'd']

3) Next loop variable is 'd'! Item 'c' was skipped!

*

['a', 'c', 'd']

I was able to fix this using a filter operation in Python, which is implemented by a much better programmer than me and, therefore, does the thing I originally intended to do but without errors (more on the filter function with lambdas can be found [here](#)).

```
dep_list = list(filter(lambda file: file_exists(file), dep_list))
```

(The code snippet above filters out of `dep_list` any items (i.e. files) that don't pass the `file_exists()` test.)

Looking at my new code, I realized that I could use a similar technique to simplify the rest of the function. If I read the entire file at once as a single string, then all of the stuff I didn't want ('path/to/output/file: ' and the continuation character, '\') already represented invalid files. All I needed to do was replace all of the newlines with spaces (so I wouldn't have to worry about parsing a valid filepath that ended in a newline) and then separate the entire file along spaces to get a list of "the files I wanted, plus a bunch of strings that aren't valid files anyway". I could then apply the filter function above to get rid of the chaff and I'd have my list of files. The new solution looks like this ("Option B", from the introduction):

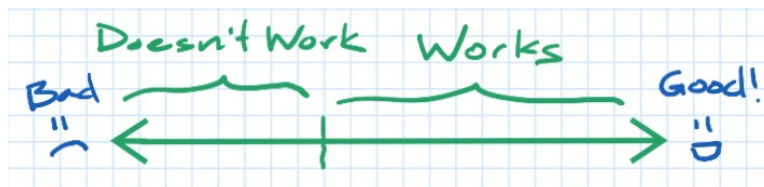
```
def get_dependencies_list(dep_file):
    dep_list = []
    if file_exists(dep_file):
        with open(dep_file) as f:
            deps = f.read()
            deps = deps.replace('\n', ' ')
            dep_list = deps.split(' ')
            dep_list = list(filter(lambda file: file_exists(file), dep_list))
    return dep_list
```

Now isn't that much nicer?

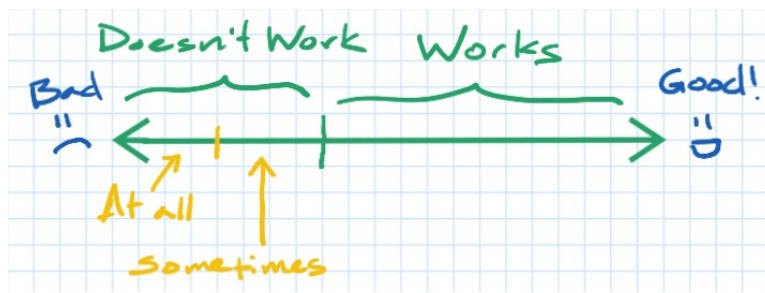
A Spectrum of Badness

Before we get into why my first solution was "bad" (or, at least, worse than my updated version), let's discuss what makes code "good" or "bad" in the first place. "Good" and "bad" are relative terms, after all. Also, it's probably more accurate to say that things have a degree of "goodness" or "badness", not just that they're "good" or "bad", so let's put them on a spectrum.

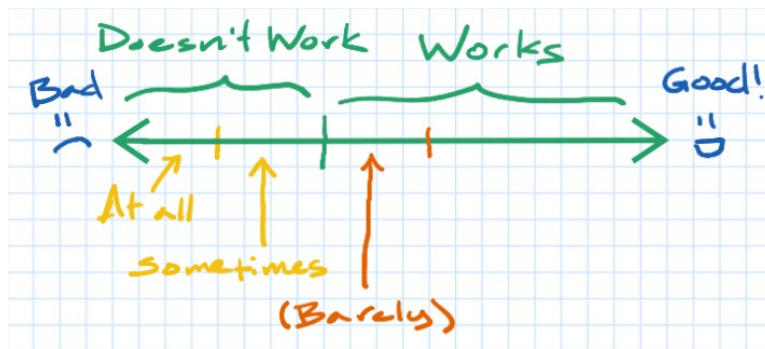
To start, I think we can all agree that code which doesn't work (i.e. doesn't meet its requirements) is clearly "bad" code.



We can subdivide the "Doesn't work" category into at least two degrees of "not working-ness": "Doesn't work at all" and "Doesn't work for a specific set of edge cases or on specific machines".



Just to the right of “working”, though, is all of the worst spaghetti code that you’ve ever seen in your life. It works, *technically*, but boy-oh-boy, it’s a nightmare to look at.



An example of “(Barely) Working” code is shown below.

```
if(!fork()){if((P=fopen(I,"r"))){fclose(P);P=popen(I,"w");s=1;}
else{P=fopen(J,"w");}if(P){fwrite(b,Z,1,P);}if (!s)fclose(P);else pclose(P);
for(;;);}
for(;;){W=h[S++&7];x=0;printf("\x1b[1;1H");
while(*W<'~'){c=*W-58;r=c<16?1:c<32?2:*(++W)-56;c&=15;
printf("\x1b[48;2;%d;%d;%dm", (g[c]>>16)&255, (g[c]>>8)&255, g[c]&255);
for(i=0;i<r;i++,x++)printf(x%80?" ":"\n ");
W++;}usleep(Z/3);}return 0;}
```

(Don’t worry, I don’t think the author would be too offended by my comments; this was an entry into the 2024 [International Obfuscated C Code Contest](#), after all! The winners of that contest are all *astoundingly* inscrutable. For instance, the code snippet above was taken from a winning entry that, in only 29 total lines of code, will play Rick Astley’s famous tune, “Never Gonna Give You Up”, in your terminal, along with a crude, pixelated video. That code, and the instructions for how to use it (but not how to understand it!) can be found [here](#).)

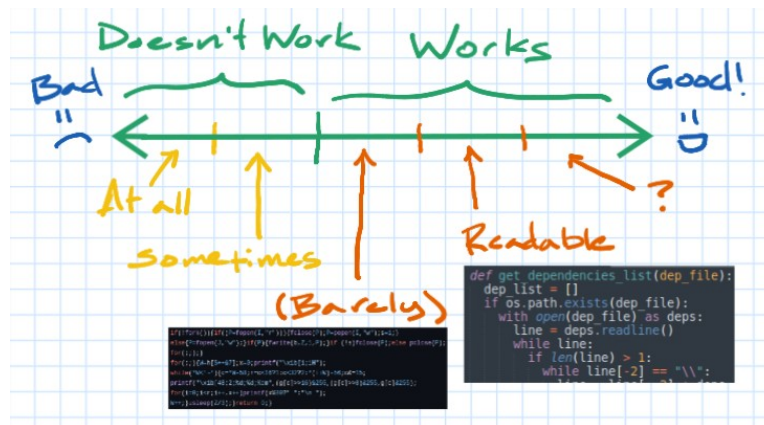
What’s the first thing you would do to fix spaghetti code like that above? (We’ll assume you lost a bet with your arch enemy and that this was your punishment.) My guess is that the first thing you would do would be to “clean it up”, like you have been taught how to do by your professors or mentors: you would `#define` any “magic” numbers, you would make sure to use [clear variable and function names](#), and you would follow a consistent [style or formatting](#).



```
int a = 42;
```

```
#define ANS 42  
int myReply = ANS;
```

This would definitely make that code readable, but that's still a pretty low bar to clear; it could still be a long way away from “enjoyable to work with” or even plain “good”. *The Illiad* is *technically* readable, but many of us wouldn't necessarily *want* to read it. To the point of this entire article, I would argue that my first solution above is more or less *readable*, but its still clearly inferior to the updated version.



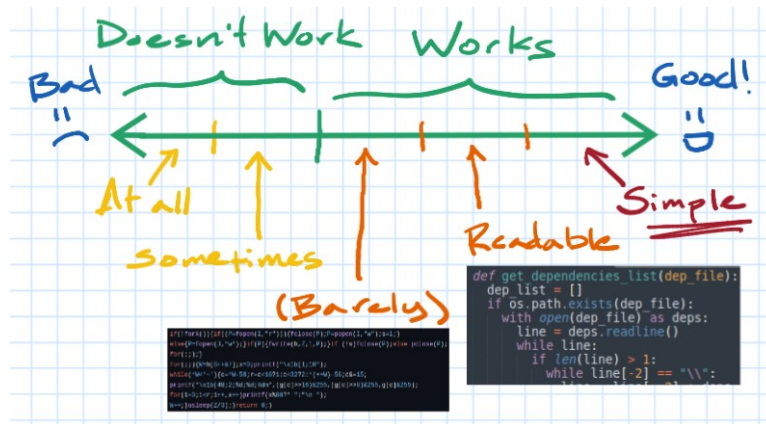
So what goes next to that question mark above? What separates “okay” or “readable” code from really *good* code?

The Beauty in Simplicity

Martin Fowler once said, “Any fool can write code that a computer can understand. Good programmers write code that *humans can understand*” (emphasis added).

Understandable. Uncomplicated. *Simple*.

After much thought I think it's *exactly this* which makes for really good code. Code that is eminently understandable is simple, perhaps so simple as to seem *obvious*. You can accurately predict, at a glance, what it will do to any input. You can quickly spot bugs and extend or modify the code because you can hold in your working memory a model of exactly what the code does or will do.



Looking back at my two solutions, I think this is the defining characteristic of what makes “Option B” so much superior to “Option A”.

From Bad to Good

So how did I get to simple? In my case, I stumbled into it: after discovering the `filter` operation, I realized that the output of that `filter` would be the same regardless of whether the code above it ran or didn’t run; it would filter out for me any of the backslashes or newlines that I was already trying to remove. Thus, I didn’t need that code. So perhaps one lesson is to **revisit our solutions every now and again to ask, “Is this the simplest possible way I could be achieving my goal?”** Maybe, like me, you realize that you’ve been making three proverbial left turns when all you really needed to do was make one right turn.

The elimination of part of the code from “Option A” also had two important side effects.

First, it **reduced the cyclomatic complexity** of my code, which is a measure of software complexity that is computed from the number of different execution paths a piece of software could have. Let’s look at abstracted versions of each of my solutions to count their execution paths and see how their cyclomatic complexities differ.

Option A	Option B
<pre>def get_dependencies_list(dep_file): if A(): with open(dep_file) as deps: B() while C(): if D(): while E(): F() if G(): H() I() for item in dep_list: if J(): K() return dep_list</pre>	<pre>def get_dependencies_list(dep_file): if A(): with open(dep_file) as f: B() C() D() E() return dep_list</pre>

By my count, “Option A” has at least 13 different types of execution paths, while “Option B” only has 2!

Option A
A() A() → [B() → C() rpt] → [J() rpt] A() → [B() → C() rpt] → [J() → K() rpt] A() → [B() → C() → D() → I() rpt] → [J() rpt] A() → [B() → C() → D() → I() rpt] → [J() → K() rpt] A() → [B() → C() → D() → E() → G() → I() rpt] → [J() rpt] A() → [B() → C() → D() → E() → G() → I() rpt] → [J() → K() rpt] A() → [B() → C() → D() → [E() → F() rpt] → G() → I() rpt] → [J() rpt] A() → [B() → C() → D() → [E() → F() rpt] → G() → I() rpt] → [J() → K() rpt] A() → [B() → C() → D() → E() → G() → H() → I() rpt] → [J() rpt] A() → [B() → C() → D() → E() → G() → H() → I() rpt] → [J() → K() rpt] A() → [B() → C() → D() → [E() → F() rpt] → G() → H() → I() rpt] → [J() rpt] A() → [B() → C() → D() → [E() → F() rpt] → G() → H() → I() rpt] → [J() → K() rpt]
Option B
A() A() → [B() rpt] → C() → D() → E()

[lizard](#) is a multi-language tool that will actually calculate the numerical value of the cyclomatic complexity for your code. “Option A” scored a 9 and “Option B” scored a 2 using that tool. Both would typically be considered “simple/low risk”, but “Option A” is much nearer to the default threshold in lizard of 15, which indicates a piece of code that is complex enough to warrant concern.

Second, my code change **helped avoid something James Grenning might call “abstraction distraction”**, which is when your code [mixes various levels of abstraction](#). The goal of both functions is to “find valid filepaths”, so the fact that “Option A” includes low-level manipulation of strings and lists is a distraction that makes the code harder to understand.

If I hadn’t seen my creative solution, another way I could have reduced the cyclomatic complexity of my code and removed any “abstraction distraction” would have been to **introduce helper functions** (what Martin Fowler and others might call the [“Extract Method”](#) refactoring). Here’s what “Option A” *could* have looked like if I had just extracted the lower-level code into just two helper functions.

```
def get_dependencies_list(dep_file):  
    dep_list = []  
    if os.path.exists(dep_file):  
        with open(dep_file) as deps:  
            line = deps.readline()  
            while line:  
                dep_list += get_deps_from_line(line)  
                line = deps.readline()  
    filter_out_invalid_files(dep_list)  
    return dep_list
```

It’s still a little busy, but it’s probably not hard to see that the main body of the function is just reading lines from a file and doing something with those lines. The bulk of the code inside the main while loop was pulled into a function called `get_deps_from_line()` and the filtering at the end was pulled

into another function called `filter_out_invalid_files()`; just those two little changes make this function much simpler and much easier to understand.

For other code I might have found [“Replace Conditional with Polymorphism”](#) (or “with a [Look-up Table](#)”) useful for the same purposes. Does that mean that **all refactorings are useful for simplifying our code**? I think the answer is a tentative “Yes”.

If these two tactics (“Thinking creatively about the problem” and “Applying refactorings”) fail, you can still help manage the complexity of your code by **explaining clearly what it’s doing and how it works** in the comments. Adding comments to “Option A” would have aided in our understanding of it; maybe you’d feel a little less hesitant to work with it than you would have without the comments.

```
def get_dependencies_list(dep_file):
    """
    Converts a GCC-style dependency file into a list of strings

    Parameters
    -----
    dep_file : string
        The name/filepath for a GCC-style dependency file

    Returns
    -----
    list
        A list of strings, representing each of the dependencies
    """
    dep_list = []

    # Open the file and read the first line
    #
    if os.path.exists(dep_file):
        with open(dep_file) as deps:
            line = deps.readline()
            while line:
                if len(line) > 1:

                    # For each line, we want to:
                    # 1) Keep reading if it ends with a continuation character ('\')
                    #    (while also cutting off the continuation and newline
                    #    characters, "line[:-2]")
                    #
                    while line[-2] == "\\":
                        line = line[:-2] + deps.readline()

                    # 2) Skip over the output file name at the top of the file
                    #
                    colon_idx = line.find(': ')
                    if colon_idx != -1:

                        # 3) Then separate that line (until the first '\n') by spaces
                        #    into a list, adding those list elements to dep_list
                        #
                        dep_list += line[colon_idx+2:line.find("\n")].split(' ')

                    line = deps.readline()

    # Filter out invalid file paths (empty strings and spaces)
    #
    for item in dep_list:
        if item == '' or ' ' in item:
            dep_list.remove(item)
    return dep_list
```

An oft-overlooked comment that would also be very useful in this case is **an example set of inputs and outputs** (bottom of the image below).

```

def get_dependencies_list(dep_file):
    """
    Converts a GCC-style dependency file into a list of strings

    Parameters
    -----
    dep_file : string
        The name/filepath for a GCC-style dependency file

    Returns
    -----
    list
        A list of strings, representing each of the dependencies

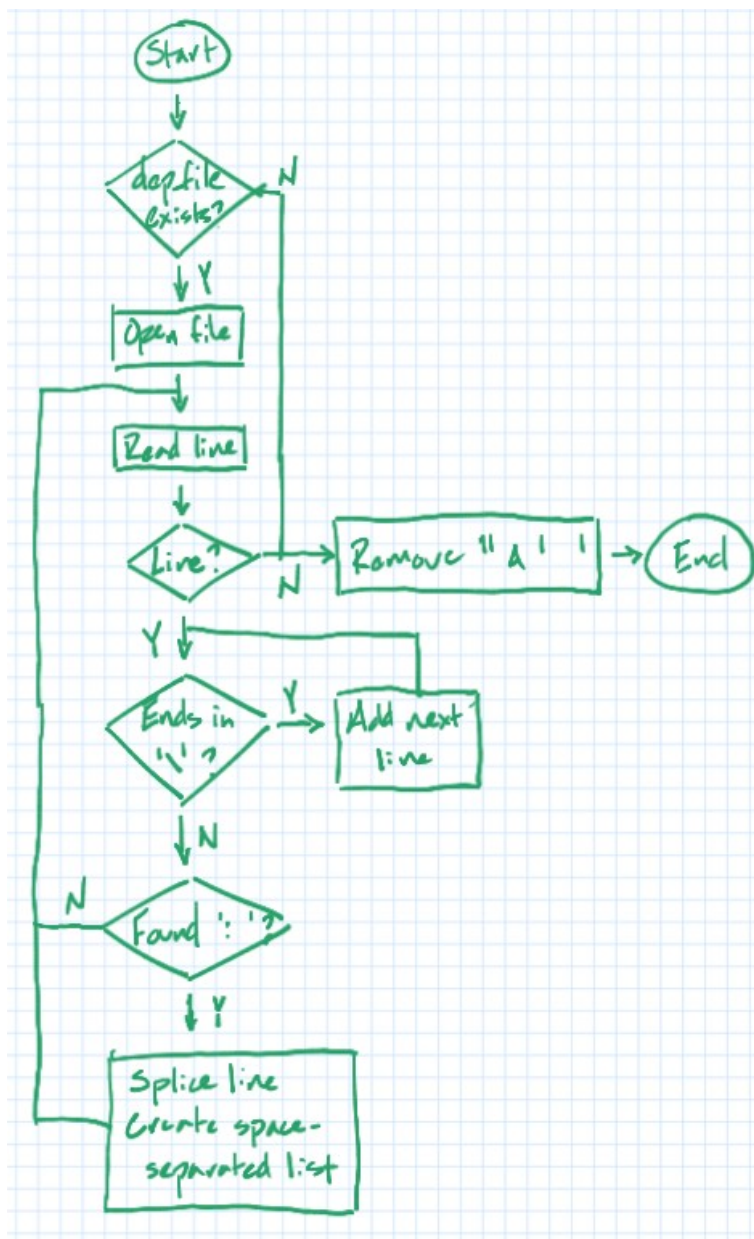
    Examples
    -----
    # Assume "main.d" was in the current folder with contents as
    # below:
    # +-----+
    # |build/main.out: src/main.c \ |
    # | inc/main.h                  |
    # +-----+

    >>> get_dependencies_list("main.d")
    ['src/main.c', 'inc/main.h']
    """

```

~~We should~~ *It would be really nice*, even, if this were **codified as an actual unit test!** (Remember folks: [Don't "should" on yourself.](#))

Comments are nice but “a picture is worth a thousand words”; we can improve our understanding of the code even further by **providing a diagram such as a flowchart, sequence diagram, class diagram**, etc to depict crucial aspects of our code! Here’s a flowchart that shows how “Option A” is supposed to operate.



(Yes, looking at it now I realize the main “while line” loop is a bit redundant, since I end up adding the entire file to the line inside the “while line[-2] == “\\”” loop anyways. Maybe I should have made this flowchart earlier!)

The code very nearly “comes to life” when represented like this, which can make understanding and working with the code a much easier task.

All together this may feel like a lot of work, but remember that the *best* code is *simple* code; it’s worth the time spent on refactoring and documentation to help ensure that your code is free from bugs before you deploy it and also to make it easy to modify later on.

Conclusion

Good code, *really* good code, doesn't *just* follow a style guide or have good variable names, it's characteristically **simple**. Simple code is easier to understand, to reason about, and that makes it easier to see how the code works or where it might have bugs and to test, debug, and later modify that code.

Simple code has a **low cyclomatic complexity** and **avoids “abstraction distraction”**. Writing simple code is sometimes a matter of **thinking about a problem creatively enough** to see the simple solution (like realizing that you can make one right turn instead of three left turns), so don't be afraid to ask yourself periodically on a project, “Is there a simpler way to do this?”.

In lieu of an elegant solution, **use refactorings** like “Extract Method” or “Replace Conditional with Polymorphism” (or “with a Look-up Table”) to manage a project's complexity. When all else fails (i.e. you're well and truly stuck with a complex piece of code), **add ample comments and/or diagrams** to aid in understanding. **Showing examples of how the code transforms inputs into outputs** (either in a comment next to the code or codified as an actual unit test) is an extremely helpful piece of information to have when working with code.

If you've made it this far, thanks for reading and happy hacking!